

Corso di Laurea triennale in Ingegneria e Scienze Informatiche

Image Denoising Technique with Neural Network

Relatore:

Prof. Lazzaro Damiana

Co/Contro Relatore

Dott. Ezio Greggio

Candidato:

Matteo Vanni

Matricola: 0000935584

Contents

1	Articolanzio	2
1.1	Panoramica sul problema	2
1.2	Utilizzo di modelli di Deep Learning	2
1.3	Dataset utilizzati	2
1.3.1	Tensorflow: Oxford-IIIT Pet	2
1.3.2	Kvasir dataset	3
2	Rumore	4
3	Decsrizione delle reti	5
3.1	Autoencoder	5
3.2	RIDNet	7
4	Analisi ed ottimizzazione	11
4.1	Prestazioni	11
4.2	Analisi dei modelli	12
4.2.1	Autoencoder	12
4.2.2	RIDNet	13
4.3	Prima run	14
4.3.1	Autoencoder	14
4.3.2	RIDNet	16
4.4	Kvasir dataset	17
4.5	Terza run	17
4.6	Quarta run	17
4.7	Quantizzazione dei modelli	17
4.7.1	Performance su dataset sconosciuti	17

1 Articolanzio

1.1 Panoramica sul problema

L'immagine denoising è il processo di rimozione di rumore da un'immagine.

Il rumore, che è causato da svariate fonti, quali foto fatte in condizioni di scarsa illuminazione o problemi che corrompono i file, causa perdita d'informazione sull'immagine.

Cos'è il rumore? Un'aggiunta casuale di pixel che non appartengono all'immagine originale e ce ne sono di varie tipologie:

Impulse Noise(IN) dove i pixel sono completamente diversi da quelli attorno. Esistono due categorie di IN: Salt and Pepper Noise(SPN) e Random Valued Impulse Noise(RVIN).

Additive White Gaussian Noise(AWGN) cambia ogni pixel dall'originale di una piccola quantità.

1.2 Utilizzo di modelli di Deep Learning

È essenziale rimuovere il rumore e ristabilire l'immagine originale dove riottenere l'immagine originale è importante per prestazioni robuste o ricostruire le informazioni mancanti è molto utile, come immagini astronomiche di oggetti molto lontani.

Le reti neurali convoluzionali lavorano bene con le immagini e ne utilizzeremo N, menzionate in alcuni paper di ricerca e compareremo i risultati di ogni modello.

1.3 Dataset utilizzati

1.3.1 Tensorflow: Oxford-IIIT Pet

Il primo dataset utilizzato per l'addestramento dei modelli è stato Oxford-IIIT Pet, fornito da TensorFlow[1]. La scelta è stata motivata dalla natura delle immagini contenute, prevalentemente raffiguranti animali, le quali presentano un alto livello di dettaglio rispetto a immagini generiche. Questa caratteristica risulta particolarmente utile nel contesto della ricostruzione di immagini degradate, poiché l'aggiunta di rumore comporta una maggiore complessità nel processo di recupero dell'informazione visiva. Inoltre, il dataset consente l'utilizzo di risoluzioni superiori a 32×32 pixel, permettendo così l'addestramento e la valutazione dei modelli su immagini di dimensioni maggiori. dataset(colonscopie).

1.3.2 Kvasir dataset

Il secondo dataset impiegato per valutare l'affidabilità e la robustezza delle reti neurali è il dataset Kvasir[2], il quale comprende 1.000 immagini endoscopiche ad alta risoluzione, in particolare raffiguranti polipi e altre anomalie del tratto gastrointestinale. Sviluppato per supportare la ricerca nel campo della diagnosi assistita da computer (CADx), con l'obiettivo di migliorare l'accuratezza e la tempestività dell'identificazione di patologie gastrointestinali.

2 Rumore

Il rumore in un'immagine rappresenta un'alterazione dei valori dei pixel che si discosta dall'informazione visiva originaria. Esso può essere introdotto in diverse fasi del processo di acquisizione, trasmissione o compressione, ed è solitamente causato da limitazioni fisiche dei sensori, fluttuazioni ambientali o errori di quantizzazione. Il rumore compromette la qualità visiva e l'integrità semantica dell'immagine, rendendo più complessi compiti computazionali quali la classificazione, la segmentazione o la ricostruzione. Tra i modelli di rumore più comuni si annoverano il rumore gaussiano (additivo e a distribuzione normale), il rumore impulsivo (salt-and-pepper), e il rumore Poissoniano (associato alla statistica fotonica). La modellazione e la gestione del rumore sono elementi centrali nella computer vision, in particolare nei contesti di image denoising, compressione con perdita e generazione di immagini sintetiche in scenari controllati.

La prima funzione di rumore è u'aggiunta di rumore casuale a ogni pixel dell'immagine.

```

1 def add_noise(x, noise_factor=0.1):
2     """
3     x: immagine in input
4     noise_factor: fattore di rumore da aggiungere
5     Ritorna:
6         x_noisy: immagine con rumore aggiunto
7     """
8     noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev
9                             ↪ =1.0)
10    x_noisy = x + noise_factor * noise
11    return x_noisy

```

E FAI RUMORE **QU** Prima funzione di rumore -i Random noise con fattore 0.3 massimo Rumore gaussiano Rumore a più layer

3 Descrizione delle reti

3.1 Autoencoder

Il primo approccio è stato quello di usare l'autencoder dell'articolo da cui sto copiando paro paro tutto.

Spiegazione del numero di layer usati e del tipo di mse loss fun, dam e learning rate di 1e-3

```
11 def build_autoencoder(input_shape):
12     """
13     Costruisce un autoencoder convoluzionale per immagini a
14         ↳ colori.
15     Parametri:
16         input_shape: forma dell'immagine in input (es. (32,32,3))
17     Ritorna:
18         autoencoder: modello compilato
19     """
20     input_img = Input(shape=input_shape)
21
22     # Encoder
23     x = Conv2D(64, (3,3), activation='relu', padding='same')(
24         ↳ input_img)
25     x = MaxPooling2D((2,2), padding='same')(x)
26     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
27     encoded = MaxPooling2D((2,2), padding='same')(x)
28
29     # Decoder
30     x = Conv2D(64, (3,3), activation='relu', padding='same')(
31         ↳ encoded)
32     x = UpSampling2D((2,2))(x)
33     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
34     x = UpSampling2D((2,2))(x)
35     decoded = Conv2D(3, (3,3), activation='sigmoid', padding='
36         ↳ same')(x)
37
38     autoencoder = Model(input_img, decoded)
39     autoencoder.compile(optimizer='adam', loss='
40         ↳ binary_crossentropy', metrics=['accuracy'])
41
42     return autoencoder
43
44 autoencoder = build_autoencoder(input_shape=(img_size[0],
45     ↳ img_size[1], 3))
```

```
40 autoencoder.summary()
```

3.2 RIDNet

stessa cosa dell'autencoder (vedere se aggiungere tutto il codice gigante del setup)

```

41 # MeanShift: sottrae (o aggiunge) la media RGB
42 @register_keras_serializable("MeanShift")
43 class MeanShift(Layer):
44     def __init__(self, rgb_mean, sign=-1, **kwargs):
45         """
46         rgb_mean: tupla con la media dei canali R, G, B.
47         sign: -1 per sottrarre la media, 1 per aggiungerla.
48         """
49         super(MeanShift, self).__init__(**kwargs)
50         self.rgb_mean = tf.constant(rgb_mean, dtype=tf.float32)
51         self.sign = sign
52
53     def call(self, x):
54         """
55         x      atteso in formato (batch, altezza, larghezza, 3)
56         """
57         return x + self.sign * self.rgb_mean
58
59 # BasicBlock: Conv2D seguita da attivazione ReLU
60 @register_keras_serializable("BasicBlock")
61 class BasicBlock(Layer):
62     def __init__(self, out_channels, kernel_size=3, stride=1,
63         ↪ use_bias=False, **kwargs):
64         super(BasicBlock, self).__init__(**kwargs)
65         self.conv = Conv2D(out_channels, kernel_size, strides=
66             ↪ stride,
67                             padding='same', use_bias=use_bias)
68         self.relu = ReLU()
69
70     def call(self, x):
71         return self.relu(self.conv(x))
72
73 # ResidualBlock: due convoluzioni con skip connection
74 @register_keras_serializable("ResidualBlock")
75 class ResidualBlock(Layer):
76     def __init__(self, out_channels, **kwargs):
77         super(ResidualBlock, self).__init__(**kwargs)
78         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
79             ↪ same')
80         self.relu = ReLU()
81         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
82             ↪ same')
83
84     def call(self, x):
85         residual = self.conv1(x)
86         residual = self.relu(residual)
87         residual = self.conv2(residual)

```



```

84         out = Add()(x, residual])
85         return ReLU()(out)
86
87 # EResidualBlock: versione estesa con convoluzioni a gruppi
88 @register_keras_serializable("EResidualBlock")
89 class EResidualBlock(Layer):
90     def __init__(self, out_channels, groups=1, **kwargs):
91         super(EResidualBlock, self).__init__(**kwargs)
92         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
93             ↪ same', groups=groups)
94         self.relu = ReLU()
95         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
96             ↪ same', groups=groups)
97         self.conv3 = Conv2D(out_channels, 1, strides=1, padding='
98             ↪ valid')
99
100     def call(self, x):
101         residual = self.conv1(x)
102         residual = self.relu(residual)
103         residual = self.conv2(residual)
104         residual = self.relu(residual)
105         residual = self.conv3(residual)
106         out = Add()(x, residual])
107         return ReLU()(out)
108
109 # Merge_Run_dual: due rami convoluzionali con dilatazioni diverse
110     ↪ , poi merge e skip connection
111 @register_keras_serializable("MergeRunDual")
112 class MergeRunDual(Layer):
113     def __init__(self, out_channels, **kwargs):
114         super(MergeRunDual, self).__init__(**kwargs)
115         # Primo ramo
116         self.conv1a = Conv2D(out_channels, 3, strides=1, padding='
117             ↪ same')
118         self.relu1a = ReLU()
119         self.conv1b = Conv2D(out_channels, 3, strides=1, padding='
120             ↪ same', dilation_rate=2)
121         self.relu1b = ReLU()
122         # Secondo ramo
123         self.conv2a = Conv2D(out_channels, 3, strides=1, padding='
124             ↪ same', dilation_rate=3)
125         self.relu2a = ReLU()
126         self.conv2b = Conv2D(out_channels, 3, strides=1, padding='
127             ↪ same', dilation_rate=4)
128         self.relu2b = ReLU()
129         # Gogeta
130         self.conv3 = Conv2D(out_channels, 3, strides=1, padding='
131             ↪ same')
132         self.relu3 = ReLU()

```

```

125     def call(self, x):
126         branch1 = self.relu1a(self.conv1a(x))
127         branch1 = self.relu1b(self.conv1b(branch1))
128
129         branch2 = self.relu2a(self.conv2a(x))
130         branch2 = self.relu2b(self.conv2b(branch2))
131
132         merged = Concatenate()([branch1, branch2])
133         merged = self.relu3(self.conv3(merged))
134         return Add()([merged, x])
135
136 # CAlayer: Channel Attention Layer
137 @register_keras_serializable("CAlayer")
138 class CAlayer(Layer):
139     def __init__(self, channel, reduction=16, **kwargs):
140         super(CAlayer, self).__init__(**kwargs)
141         self.channel = channel
142         self.reduction = reduction
143         self.conv1 = Conv2D(channel // reduction, 1, strides=1,
144             ↳ padding='same')
145         self.relu = ReLU()
146         self.conv2 = Conv2D(channel, 1, strides=1, padding='same'
147             ↳ , activation='sigmoid') #sigmoid: risultato tra 0 e
148             ↳ 1
149
150     def call(self, x):
151         # Calcolo della media globale per canale
152         y = tf.reduce_mean(x, axis=[1, 2], keepdims=True)
153         y = self.relu(self.conv1(y))
154         y = self.conv2(y)
155         return Multiply()([x, y])
156
157 # Block: combinazione di MergeRunDual, ResidualBlock,
158     ↳ EResidualBlock e CAlayer
159 @register_keras_serializable("Block")
160 class Block(Layer):
161     def __init__(self, out_channels, **kwargs):
162         super(Block, self).__init__(**kwargs)
163         self.merge_run_dual = MergeRunDual(out_channels)
164         self.residual_block = ResidualBlock(out_channels)
165         self.e_residual_block = EResidualBlock(out_channels)
166         self.ca = CAlayer(out_channels)
167
168     def call(self, x):
169         r1 = self.merge_run_dual(x)
170         r2 = self.residual_block(r1)
171         r3 = self.e_residual_block(r2)
172         out = self.ca(r3)
173         return out

```

```

171 # Creazione del modello
172 @register_keras_serializable("RIDNET")
173 def RIDNET(n_feats=64, rgb_range=1.0):
174     """
175     n_feats: numero di feature channels usate all'interno del
176             ↳ modello.<br>
177     rgb_range: scala dei valori RGB (ad es. 1.0 se l'input
178             ↳ normalizzato [0,1]).
179     """
180     rgb_mean = (0.4488, 0.4371, 0.4040)
181
182     input_layer = Input(shape=(None, None, 3))
183
184     # Sottosottrai la media (MeanShift con sign=-1)
185     sub_mean = MeanShift(rgb_mean, sign=-1)
186     x = sub_mean(input_layer)
187
188     # Testa: BasicBlock (conv + ReLU)
189     head = Conv2D(n_feats, 3, strides=1, padding='same',
190                 ↳ activation='relu')(x)
191
192     # Serie di blocchi
193     b1 = Block(n_feats)(head)
194     b2 = Block(n_feats)(b1)
195     b3 = Block(n_feats)(b2)
196     b4 = Block(n_feats)(b3)
197
198     # Coda: convoluzione finale per ottenere 3 canali
199     tail = Conv2D(3, 3, strides=1, padding='same')(b4)
200
201     # Aggiungi la media (MeanShift con sign=+1)
202     add_mean = MeanShift(rgb_mean, sign=1)
203     res = add_mean(tail)
204
205     # Connessione residua a livello di immagine: somma con l'
206             ↳ input originale
207     output = Add()([res, input_layer])
208
209     model = Model(inputs=input_layer, outputs=output)
210     return model
211
212 model = RIDNET(n_feats=32, rgb_range=1.0)
213 model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])
214             ↳ )
215 model.summary()

```

4 Analisi ed ottimizzazione

4.1 Prestazioni

PSNR è il metodo più comune per misurare la qualità delle immagini.

Il PSNR è definito come il rapporto tra il massimo valore possibile di un segnale e il valore del rumore che disturba la qualità della sua rappresentazione.

Normalmente misurato in una scala logaritmica in decibel(dB).

Data l'immagine originale(g) e l'immagine rumorosa(f), il PSNR è definito come:

$$PSNR = 20 \log_{10} \left(\frac{MAX_f}{\sqrt{MSE}} \right)$$

dove MAX_f è il valore massimo del pixel dell'immagine e si calcola come

$$MSE = \frac{1}{mn} \sum_0^{m-1} \sum_0^{n-1} ||f(i, j) - g(i, j)||^2$$

mentre MSE (*Mean Square Error*) è l'errore quadratico medio tra l'immagine originale e quella rumorosa.

4.2 Analisi dei modelli

4.2.1 Autoencoder

La rete neurale è un autoencoder convoluzionale simmetrico progettato per la compressione e successiva ricostruzione di immagini RGB.

Nella fase di encoding, l'immagine in input viene inizialmente processata da un primo strato convoluzionale (Conv2D) con 64 filtri, ciascuno di dimensione 3×3 , seguito da un'operazione di sottocampionamento (MaxPooling2D) che riduce la risoluzione spaziale da 112×112 a 56×56 mantenendo invariato il numero di canali. Un secondo strato convoluzionale, anch'esso composto da 64 filtri, approfondisce l'estrazione delle caratteristiche locali, cui segue un ulteriore livello di pooling che porta la risoluzione a 28×28 pixel. A questo punto, l'informazione visiva è compressa in una rappresentazione latente di dimensione (28, 28, 64), che mantiene un buon bilanciamento tra capacità rappresentativa e riduzione della dimensionalità.

La fase di decoding ha l'obiettivo di ricostruire l'immagine originale a partire dalla rappresentazione compressa. In essa, un ulteriore strato convoluzionale affina la mappa delle caratteristiche, seguito da un'operazione di upsampling (UpSampling2D) che ripristina la dimensione spaziale a 56×56 .

Successivamente, un altro strato convoluzionale migliora la qualità della ricostruzione a tale scala. Un secondo upsampling riporta l'immagine alla dimensione originale di 112×112 pixel. Infine, un ultimo strato convoluzionale con 3 filtri produce l'output finale, riconvertendo il volume delle feature maps in un'immagine RGB.

L'architettura è completamente simmetrica rispetto alla fase di encoding e decoding e non prevede strati densi (fully connected), risultando pertanto particolarmente adatta alla gestione di immagini di dimensione variabile. Inoltre, la scelta di mantenere una rappresentazione latente relativamente "larga" ($28 \times 28 \times 64$) rispetto ad autoencoder più tradizionali consente una migliore preservazione dei dettagli visivi, rendendo questo modello particolarmente adatto a compiti di denoising o ricostruzione di immagini degradate.

Architettura dell'autoencoder. Model: functional		
Layer	Output Shape	Params #
Input layer	(None, 112, 112, 3)	0
Conv2D	(None, 112, 112, 64)	1,792
MaxPooling2D	(None, 56, 56, 64)	0
Conv2D 1	(None, 56, 56, 64)	36,928
MaxPooling2D 1	(None, 28, 28, 64)	0
Conv2D 2	(None, 28, 28, 64)	36,928
UpSampling2D	(None, 56, 56, 64)	0
Conv2D 3	(None, 56, 56, 64)	36,928
UpSampling2D 1	(None, 112, 112, 64)	0
Conv2D 4	(None, 112, 112, 3)	1,731
Params		
Total params:	114,307	446.51 KB
Trainable params:	114,307	446.51 KB
Non-trainable params:	0	0.00 B

4.2.2 RIDNet

La rete RIDNet è composta da vari strati che interagiscono in modo complesso, e come si vede dalla tabella ogni passaggio è concatenato in modo sequenziale, dove l'output di un blocco è l'input del successivo ??.

Il flusso di dati inizia con l'Input Layer 5, il quale riceve un'immagine di dimensioni sconosciute (None, None, None, 3). Questo strato non ha parametri addizionali, essendo semplicemente un livello di ingresso.

Subito dopo, il Mean Shift 10 applica una trasformazione ai dati, mantenendo invariata la dimensione dell'immagine. Questo strato ha il compito di centralizzare l'intensità dei pixel.

Il primo blocco convoluzionale della rete è rappresentato dal Conv2D 250, che riduce progressivamente la dimensione spaziale dell'immagine mentre aumenta il numero di filtri da 3 a 32.

Successivamente, il flusso prosegue attraverso una serie di Block che non solo applicano operazioni convoluzionali ma anche attivazioni non lineari, garantendo una maggiore capacità di rappresentazione della rete. Ogni blocco (da Block 20 a Block 23) mantiene una struttura simile e continua ad applicare operazioni convoluzionali mantenendo gli stessi parametri.

L'output dell'ultimo blocco convoluzionale, il Conv2D 299, produce un'immagine di output della stessa dimensione dell'immagine di ingresso, con qualche parametro in meno e nuovamente 3 canali, questa è una prima previsione dell'immagine finale.

Infine, un ulteriore strato di Mean Shift 11 viene applicato per eseguire un altro aggiustamento del valore medio dei pixel nell'immagine di output. Anche in questo caso, non vengono introdotti nuovi parametri.

L'output finale della rete è generato dallo strato Add 413, che esegue una somma tra l'output di Mean Shift 11 e il risultato precedente. Questo step finale non introduce parametri addizionali, ma è cruciale per combinare le informazioni passate attraverso la rete, migliorando la qualità del risultato finale.

In sintesi, la RIDNet si basa su una sequenza di operazioni convoluzionali seguite da trasformazioni di "mean shift" e operazioni di somma, ottimizzando la qualità dell'immagine di output attraverso una struttura profonda e iterativa. La rete è progettata per ridurre progressivamente l'immagine, applicare convoluzioni e migliorare la qualità attraverso un processo di addizione che incorpora informazioni provenienti da differenti livelli della rete.

Architettura della RIDNet. Model: functional_5			
Layer (type)	Output Shape	Param #	Connected to
input_layer_5	(None, None, None, 3)	0	-
mean_shift_10	(None, None, None, 3)	0	input_layer_5[0][0]
conv2d_250 (Conv2D)	(None, None, None, 32)	896	mean_shift_10[0][0]
block_20 (Block)	(None, None, None, 32)	93,666	conv2d_250[0][0]
block_21 (Block)	(None, None, None, 32)	93,666	block_20[0][0]
block_22 (Block)	(None, None, None, 32)	93,666	block_21[0][0]
block_23 (Block)	(None, None, None, 32)	93,666	block_22[0][0]
conv2d_299 (Conv2D)	(None, None, None, 3)	867	block_23[0][0]
mean_shift_11	(None, None, None, 3)	0	conv2d_299[0][0]
add_413 (Add)	(None, None, None, 3)	0	mean_shift_11[0][0]
Params			
Total params:	376,427	1.44 MB	
Trainable params:	376,427	1.44 MB	
Non-trainable params:	0	0.00 B	

4.3 Prima run

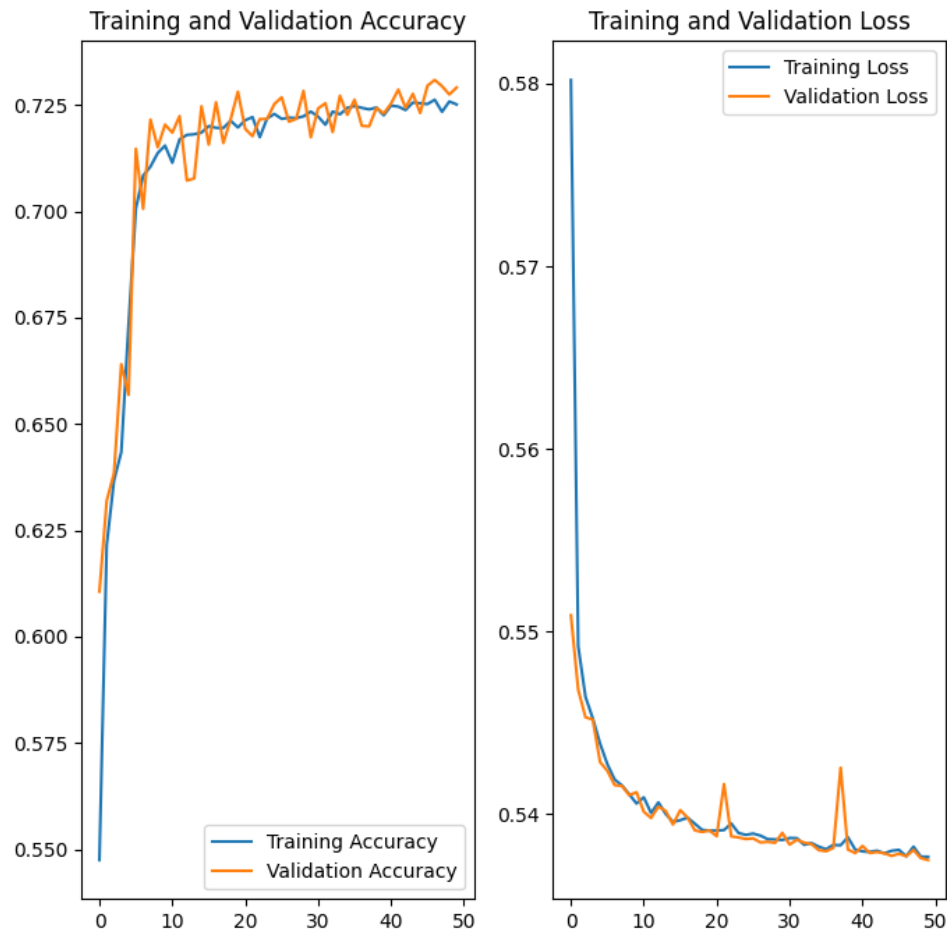
Il primo allenamento è stato sottoposto a entrambi i modelli con immagini RGB dal dataset tensorflow "oxford_iiit_pet" con risoluzione 112x112 pixel, un noise_factor di 0.4 e 50 epoche di training ed entrambe con una batch_size di 64.

Il dataset non presenta una suddivisione tra train, validation e test ma solo tra train e test, allora è stato suddiviso il training in 80% training e 20% validation.

4.3.1 Autoencoder

Una volta allenato il modello il risultato del training presenta un leggero overfitting sul validation set, mentre sempre nel validation sono presenti un paio di picchi nel loss (come da grafico 4.1

Figure 4.1: First training of the autoencoder



Poi è stata eseguita una predict e confronto del loss level tra immagini ricreate e reali con diversi livelli di rumore utilizzando la funzione di psnr from skimage.metrics import peak_signal_noise_ratio

la media PSNR sul test set è di 5.43 dB. Come si può notare dal grafico??

4.3.2 RIDNet

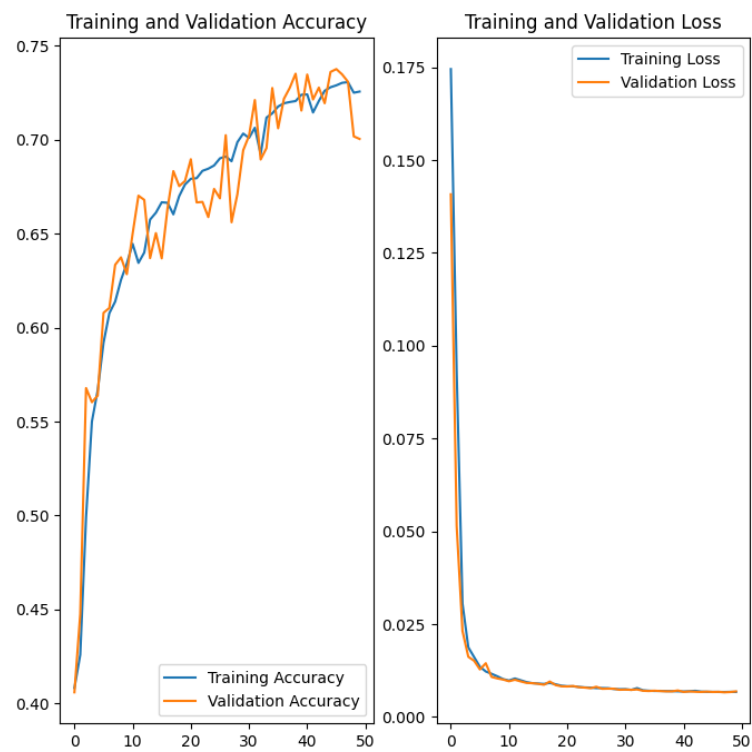


Figure 4.3: First training of the autoencoder

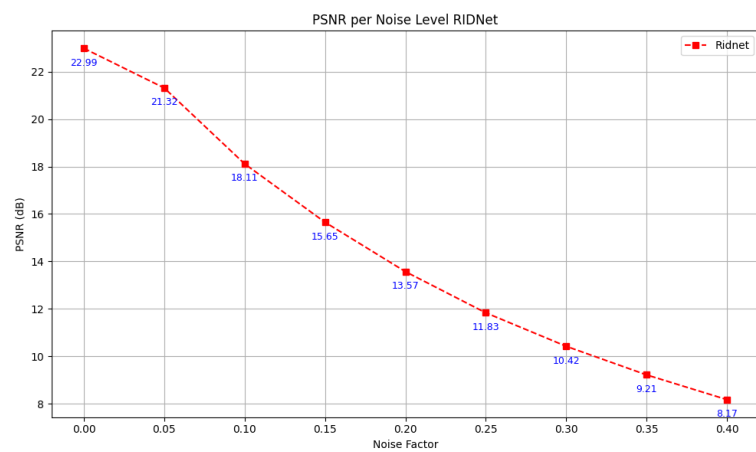


Figure 4.4: First psnr of the autoencoder

la media PSNR sul test set è di 5.37 dB. Come si può notare dal grafico

4.4 Kvasir dataset

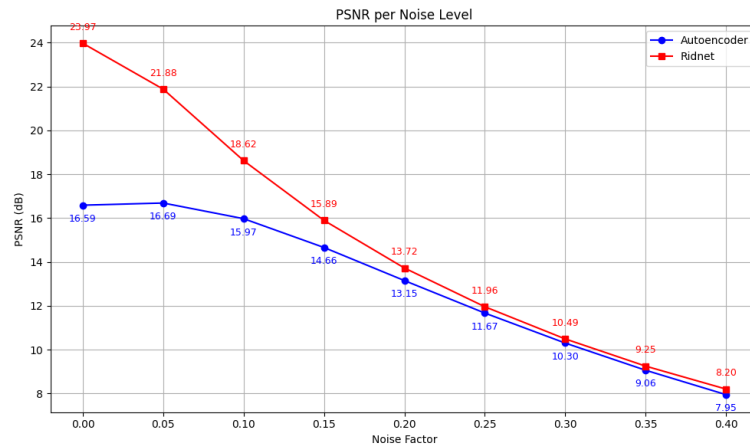


Figure 4.5: PSNR value for kvasir dataset

batch_size più piccola(?) o diverse dimensioni tra split del dataset e train; 50(?) epoche

4.5 Terza run

<QUI SAREBBE DA DOVER CAMBIARE LEARNING RATE E LOSS LEVEL>

4.6 Quarta run

<CAMBIO DELLE ARCHITETTURE DELLE DUE RETI PER MIGLIORARLA A LIVELLO DI PRESTAZIONI>

4.7 Quantizzazione dei modelli

4.7.1 Performance su dataset sconosciuti

Bibliography

- [1] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.
- [2] Konstantin Pogorelov, Kristin Ranheim Randel, Carsten Griwodz, Sigrun Losada Eskeland, Thomas de Lange, Dag Johansen, Concetto Spampinato, Duc-Tien Dang-Nguyen, Mathias Lux, Peter Thelin Schmidt, Michael Riegler, and Pål Halvorsen. Kvasir: A multi-class image dataset for computer aided gastrointestinal disease detection. In *Proceedings of the 8th ACM on Multimedia Systems Conference*, MMSys'17, pages 164–169, New York, NY, USA, 2017. ACM.